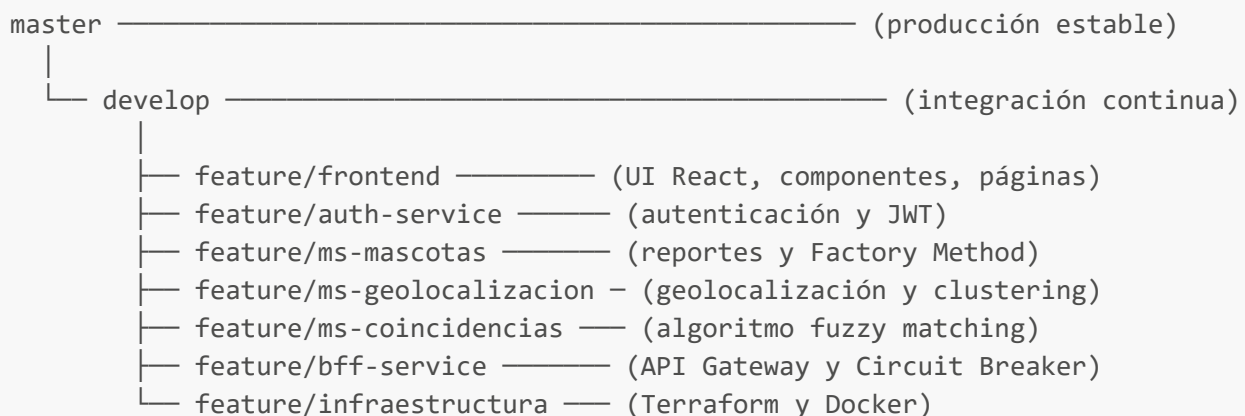


Plan de Branching — Sanos y Salvos

1. Estrategia adoptada: Git Flow adaptado

El proyecto utiliza una estrategia basada en **Git Flow** adaptada al contexto de un equipo de desarrollo académico. Se definen ramas con propósitos claros para garantizar la estabilidad del código en producción y facilitar la colaboración paralela entre integrantes del equipo.

2. Estructura de ramas



3. Descripción de cada rama

master

- **Propósito:** Código en producción, siempre estable y desplegable.
- **Política:** Solo recibe merges desde **develop** tras revisión.
- **Protección:** No se hace push directo; requiere Pull Request.

develop

- **Propósito:** Rama de integración continua. Concentra el trabajo de todas las features antes de llegar a producción.
- **Política:** Recibe merges de las ramas **feature/***.
- **Estado:** Siempre debe compilar y pasar los tests.

feature/*

- **Propósito:** Desarrollo de funcionalidades específicas de forma aislada.
 - **Convención de nombres:** **feature/<nombre-componente>** en minúsculas y con guiones.
 - **Política:** Se crean desde **develop** y se fusionan de vuelta a **develop** mediante Pull Request.
-

4. Flujo de trabajo por integrante

1. Crear rama desde develop:
`git checkout develop`
`git pull origin develop`
`git checkout -b feature/mi-funcionalidad`
2. Desarrollar y commitear:
`git add <archivos>`
`git commit -m "feat: descripción breve del cambio"`
3. Sincronizar con develop si hubo cambios:
`git fetch origin develop`
`git rebase origin/develop`
4. Subir rama y abrir Pull Request:
`git push origin feature/mi-funcionalidad`
5. Revisión y merge a develop (sin fast-forward):
`git checkout develop`
`git merge --no-ff feature/mi-funcionalidad`
`git push origin develop`
6. Eliminar rama feature:
`git branch -d feature/mi-funcionalidad`
`git push origin --delete feature/mi-funcionalidad`

5. Convención de mensajes de commit

Se sigue la especificación **Conventional Commits**:

```
<tipo>(<scope>): <descripción corta>
```

Tipos válidos:

feat	→ Nueva funcionalidad
fix	→ Corrección de bug
docs	→ Solo documentación
refactor	→ Refactorización sin cambio de comportamiento
test	→ Agregar o modificar tests
chore	→ Tareas de mantenimiento (dependencias, CI)

Ejemplos:

```
feat(ms-mascotas): implementar ReporteFactory con Factory Method  
fix(bff-service): corregir circuit breaker cuando ms-mascotas no responde  
test(ms-coincidencias): agregar pruebas unitarias a ScoringService  
docs(auth-service): agregar README con instrucciones de instalación
```

6. Gestión de conflictos

Los conflictos ocurren cuando dos ramas modifican las mismas líneas. El proceso de resolución es:

```
# 1. Identificar el conflicto al hacer merge/rebase
git merge feature/otra-rama
# → CONFLICT (content): Merge conflict in src/main/java/.../Service.java

# 2. Abrir el archivo y resolver manualmente
# Buscar marcadores:
# <<<<<< HEAD          ← versión local
# código actual
# =====
# código entrante
# >>>>>> feature/otra-rama

# 3. Dejar el código correcto, eliminar marcadores

# 4. Marcar como resuelto
git add src/main/java/.../Service.java

# 5. Continuar el merge
git commit -m "fix: resolver conflicto en Service.java entre feature/A y
feature/B"
```

Prevención: Sincronizar frecuentemente con `develop` (pasos 3 y 4 del flujo) minimiza los conflictos.

7. Evidencia de ramas y merges del proyecto

Ramas creadas durante el desarrollo

Rama	Propósito	Estado
<code>master</code>	Producción	Activa
<code>develop</code>	Integración	Activa
<code>feature/frontend</code>	Componentes React y mapa	Mergeada
<code>feature/auth-service</code>	Autenticación JWT	Mergeada
<code>feature/ms-mascotas</code>	CRUD reportes + Factory	Mergeada
<code>feature/ms-geolocalizacion</code>	PostGIS + clustering	Mergeada
<code>feature/ms-coincidencias</code>	Fuzzy matching	Mergeada
<code>feature/bff-service</code>	API Gateway	Mergeada
<code>feature/infraestructura</code>	Terraform + Docker	Mergeada

Historial de merges (resumen)

```
develop ← feature/auth-service      (merge: implementación completa de auth)
develop ← feature/ms-mascotas        (merge: Factory Method + RabbitMQ publisher)
develop ← feature/ms-geolocalizacion (merge: PostGIS + clustering algorithm)
develop ← feature/ms-coincidencias   (merge: FuzzyWuzzy + scoring strategy)
develop ← feature/bff-service        (merge: Circuit Breaker + proxy services)
develop ← feature/frontend            (merge: UI completa con Leaflet y Context)
develop ← feature/infraestructura     (merge: Terraform AWS + docker-compose)
master  ← develop                    (merge: release v1.0.0 estable)
```

8. Integración continua con GitHub Actions

El archivo `.github/workflows/ci-cd.yml` ejecuta automáticamente:

1. En cada push a `feature/*` y `develop`:

- Compilación Maven de los 5 microservicios
- Ejecución de pruebas unitarias
- Generación de reporte JaCoCo

2. En merge a `master`:

- Build de imágenes Docker
- Push a Amazon ECR
- Despliegue automático en AWS

Esto garantiza que ningún código roto llegue a las ramas principales.

9. Ventajas de esta estrategia

Ventaja	Descripción
Aislamiento	Cada integrante trabaja en su rama sin interrumpir al equipo
Trazabilidad	El historial de commits muestra qué se cambió, cuándo y por qué
Estabilidad	<code>master</code> siempre está en estado desplegable
Revisión	Los Pull Requests permiten revisión de código antes de integrar
Rollback	Si algo falla, se puede revertir un merge específico